```
100 'Floating Point Loop Control
110 FOR X = 0 TO 10 STEP 0.01
120   PRINT X
130 NEXT X
```

Program Output:

```
 0.00
 0.01
 0.02
 0.03
  ⋮
 9.960132
 9.970133
 9.980133
 9.990133
```

```
100 'Integer Loop Control
110 FOR I% = 0 TO 1000
120   X = I%/100
130   PRINT X
140 NEXT I%
```

Program Output:

```
 0.00
 0.01
 0.02
 0.03
  ⋮
 9.96
 9.97
 9.98
 9.99
10.00
```

TABLE 4-2
Comparison of floating point and integer variables for loop control. The left hand program controls the FOR-NEXT loop with a floating point variable, *X*. This results in an accumulated round-off error of 0.000133 by the end of the program, causing the last loop value, $X = 10.0$, to be omitted. In comparison, the right hand program uses an integer, *I%*, for the loop index. This provides *perfect* precision, and guarantees that the proper number of loop cycles will be completed.

Table 4-2 illustrates a particularly annoying problem of round-off error. Each of the two programs in this table perform the same task: printing 1001 numbers equally spaced between 0 and 10. The left-hand program uses the floating point variable, *X*, as the loop index. When instructed to execute a loop, the computer begins by setting the index variable to the starting value of the loop (0 in this example). At the end of each loop cycle, the step size (0.01 in the case) is *added* to the index. A decision is then made: are more loops cycles required, or is the loop completed? The loop ends when the computer finds that the value of the index is *greater than* the termination value (in this example, 10.0). As shown by the generated output, round-off error in the additions cause the value of *X* to accumulate a significant discrepancy over the course of the loop. In fact, the accumulated error *prevents* the execution of the last loop cycle. Instead of *X* having a value of 10.0 on the last cycle, the error makes the last value of *X* equal to 10.000133. Since *X* is greater than the termination value, the computer thinks its work is done, and the loop prematurely ends. This missing last value is a common bug in many computer programs.

In comparison, the program on the right uses an integer variable, *I%*, to control the loop. The addition, subtraction, or multiplication of two integers always produces another integer. This means that fixed point notation has absolutely no round-off error with these operations. Integers are ideal for controlling loops, as well as other variables that undergo multiple mathematical operations. The last loop cycle is guaranteed to execute! Unless you have some strong motivation to do otherwise, always use integers for loop indexes and counters.